

Presentation content

The nine required sections, slide by slide

LeaseManager — Final Year Project

Section	Slides
1 · Host organisation	2
2 · Problem statement	2
3 · Study and critique of existing solutions	3
4 · Proposed solution	4
5 · Functional and non-functional requirements	6
6 · Technologies used	2
7 · Design	3
8 · Progress	3
9 · Conclusion and perspectives	2
Total	27

How to use this document

One block per slide. The title is to be reused as is, the content is to be pasted into Canva. The **Say this** note does **not** go on the slide — it belongs in your speaker notes.

Figures

All taken from the database or from a test run on 3 October 2026. They are verifiable: an examiner may ask for the source.

SECTION 1

Host organisation

2 slides

Slide 1.1

The company

To be completed — this information is not in the code

Take it from the company profile or its website. Do not invent anything: your industrial supervisor will be in the room.

Slide structure

- **Company name** and logo
- **Sector** and year founded
- **Headcount** and locations
- **Core business** in one sentence
- **Market position:** which clients, which markets

Slide 1.2

Internship context

Slide structure

- **Host department** and its remit
- **Industrial supervisor:** name and role
- **Duration** and dates of the internship
- **Assignment:** design and build a multi-organisation platform for office lease management

Say this

Move through this quickly. The section is expected but earns no technical marks: two minutes is enough. End on the assignment, which leads into the problem statement.

Problem statement

2 slides

Slide 2.1

The business: leasing offices in Doha

A property company owns office towers and leases workspace to other businesses: private offices, dedicated desks, meeting rooms, event spaces.

Three groups of actors work on the same data

Actor	What they do
The landlord	Owns the buildings. Lists, reviews, approves, invoices, collects.
The renter	Occupies the space. Books, signs, pays, reports faults.
Operations staff	Reception (calls, viewings), technicians, finance.

Two rules specific to the Qatari market

- A lease is concluded **in person**: confirmation call, site visit, paper file, and only then a signature
- Rent is **not paid online**: cash, bank transfer or cheque

Slide 2.2

The question this project answers

Problem statement

*How do you build a single platform that serves **several competing property companies**, honours a face-to-face commercial process, and guarantees that no organisation ever sees another's data?*

Three difficulties to resolve

- **Isolation.** Two competitors on one installation: not a row, not a total, not a counter in common
- **Process.** Represent a face-to-face sequence that existing products ignore
- **Model.** One record belongs to two organisations by two different paths

Say this

Deliver the problem statement slowly, then pause. It is the sentence the examiners will use to judge whether everything that follows holds together.

SECTION 3

Study and critique of existing solutions

3 slides

Slide 3.1

Current practice: spreadsheets and email

Weakness	Consequence
Availability is uncertain	No single source of truth; booking conflicts surface too late
Manual collections	Due dates tracked from memory; payment delays stretch out
Faults leave no trace	No resolution time, no cost, no history per space
No audit trail	Impossible to establish who approved what, and when; a problem in a dispute

Slide 3.2

Commercial products on the market

Four mismatches against the requirement.

Limitation	Consequence
A single level of client	Does not separate who owns the walls from who occupies them → several competing landlords cannot be hosted together
Online payment assumed	Unsuited to a cash market
Fully digital intake	Ignores the confirmation call, the site visit and the paper file
Per-user pricing	Prohibitive for a company with a large operations staff

Say this

The first row is the decisive argument: it is what justifies the project existing at all. Do not treat it as one row among four.

Summary: the technical constraint

Breather slide — dark background, very short text

Two competing companies, one installation, zero shared data.

Not a row, not a total, not a counter. This requirement, rather than the feature list, is what shapes the whole architecture that follows.

In Canva: background #084449, white text, one sentence at large size. No bullets, no illustration.

Say this

The hinge between the problem and the solution. Pause for two seconds before moving on.

SECTION 4

Proposed solution

4 slides

Slide 4.1

A three-level tenancy model

The specification called for multi-tenancy at **one level**. The business requires **two, nested**.

Level	Actor	What they can see
1	Platform owner	Operates the service. The only role that reads across organisations.
2	Property company (landlord)	Buildings, floors, spaces. Lists, approves, invoices, collects. Its own portfolio only.
3	Renter company (tenant)	Books, signs, pays, reports. Its own data, plus the public listings.

Implementation: `tenants.organization_type` $\in$ { CLIENT, RENTER }

Say this

This is the project's contribution beyond the specification. Present it as analysis that reshaped the model — not as a deviation you were forced into.

Slide 4.2

The central difficulty (make this slide large)

A booking, or a maintenance ticket, belongs to **two organisations** by **two different paths**.

THE RENTER	THE LANDLORD
<code>tenant_id</code> carried on the row “I am the one who booked it”	<code>space → floor → building.tenant_id</code> “those are my walls”

Closing line for the slide

*Confusing these two paths is the origin of all **9 cross-tenant defects** found and fixed during development.*

Say this

The most important slide in the presentation. Trace both arrows out loud.

Slide 4.3**How the rule is enforced, route by route**

#	Stage	Role
1	JwtAuthGuard	Validates the token, populates the caller's identity
2	RolesGuard	Checks <code>@Roles(...)</code> against the caller's role
3	ValidationPipe	Rejects unexpected fields rather than ignoring them
4	Controller → *ForUser	Never the unscoped method
5	AccessPolicyService	Turns the caller into a Prisma filter

`isCrossTenantReader()` is true for the platform owner alone.

Slide 4.4**Deployment architecture**

Browser → nginx ↗ React SPA (built)
 ↳ NestJS API → Prisma → PostgreSQL
 ↳ Socket.IO (notifications)

4 containers · one origin · no CORS · no hostname compiled into the bundle

Illustration: docs/diagrams/09-deployment.png

SECTION 5

Functional and non-functional requirements

6 slides

Slide 5.1

Coverage of the specification

§ Spec	Module	Status
3.1	Authentication & accounts	Complete
3.2	Space management	Complete
3.3	Catalogue & pricing	Complete
3.4	Availability & bookings	Complete
3.5	Leases / contracts	Complete
3.6	Invoicing & payments	Complete
3.7	Maintenance & tickets	Complete
3.8	Notifications	Complete
3.9	Reporting & dashboards	Complete
3.10	Administration & audit	Complete

30 controllers · 218 routes · 25 data models

Functional requirements (1 of 2)

§3.1 — Authentication & accounts

- Email and password sign-in, JWT with refresh
- Multiple concurrent sessions per device
- Role-based access control: **9 roles** — the specification listed 6
- Password reset by email

§3.2 — Space management

- Buildings → Floors → Spaces (7 types)
- Features, equipment, photographs, position on a floor plan
- 5 statuses: available, occupied, reserved, maintenance, out of service

§3.3 — Catalogue & pricing

- Three rate cards: hourly, daily, monthly
- Add-on services with their own billing cycle
- **Promotion codes unique per company**: two landlords can both run the same code without collision

§3.4 — Availability & bookings

- Calendar by site, floor and space type
- **Double-booking prevention**: overlap check before the write
- Check-in and check-out
- **12 states**, including a phase absent from the specification: confirmation call → site visit → document upload

Functional requirements (2 of 2)

§3.5 — Leases / contracts

- Generated from a confirmed booking
- Security deposit: lodging and refund
- **Renewal and termination**
- Automatic expiry reminders at 60 / 30 / 7 days

§3.6 — Invoicing & payments

- Itemised invoices, configurable tax rate
- Settlement: **cash · bank transfer · cheque**
- Automatic reminders at day 1 / 7 / 14 past due
- Export to PDF and CSV/XLSX

§3.7 — Maintenance & tickets

- Reporting limited to spaces the reporter **actually occupies**
- 7 categories, 5 priorities, assignment to a technician
- Resolution cost recorded

§3.8 and §3.9 — Notifications and reporting

- In-app notifications delivered **live** over a websocket, plus email
- Occupancy rate, revenue per site, arrears, mean time to resolution
- Revenue forecast, occupancy heat map

Deliberate departures from the specification

Specification	Decision	Justification
Online payment (Stripe), <i>optional</i>	Dropped	Qatari market settles in cash — a constraint given by the industrial supervisor
Supabase (Auth + Storage)	Replaced with Passport-JWT + Prisma + Cloudinary	Supabase models one level of tenant; three-level isolation requires control over how each query is built
BullMQ + Redis, <i>optional</i>	Not adopted	Only 3 daily jobs → <code>@nestjs/schedule</code> is sufficient, one dependency fewer
QR code, recurrence, SLA	Not implemented	Marked <i>optional</i> in the specification

Added beyond the specification: 3 AI features · a public space-finding assistant · live notifications · an automated isolation audit

Say this

The slide that decides your mark: it proves the specification was read and reasoned about rather than followed blindly. Prepare the Supabase question.

Non-functional requirements

Requirement	Implementation	Evidence
JWT auth + refresh	Passport-JWT, per-device sessions	—
RBAC + tenant isolation	@Roles on every route + AccessPolicyService	43 probes
Input validation, injection safety	whitelist + forbidNonWhitelisted; Prisma parameterises every query	Unknown fields rejected
Rate limiting	ThrottlerGuard on public endpoints	—
Audit logging	append-only audit_logs (actor, IP, severity)	—
Performance	Pagination, TTL cache, request de-duplication	—
Reliability	Runs with no external keys at all ; mail degrades through 3 tiers	—
Maintainability	NestJS modules, 252 tests , CI/CD, Swagger	—
Usability	Responsive, light and dark themes, per-tab sessions	—

Slide 5.6 Proving the isolation (strong slide)

Correct isolation **partitions** the data.

Measure	Msheireb	West Bay	Lusail	Platform
Revenue collected (QAR)	5,300	11,800	10,100	27,200
Bookings	6	6	6	18
Spaces	24	24	24	72
Maintenance tickets	18	18	18	54

Conclusion

*The three portfolios add up to **exactly** the platform total. If a row leaked, or were counted twice, the sum would not close. Security becomes an arithmetic property anyone can check.*

Say this

The most convincing argument for a technical panel. Do the addition out loud on the first row.

SECTION 6

Technologies used

2 slides

Slide 6.1

One TypeScript stack, end to end

Layer	Technologies
Server	NestJS 11 · Prisma 5 · PostgreSQL 13 · Passport-JWT · Socket.IO
Client	React 19 · Vite 7 · TypeScript · Ant Design 6 · TanStack Query · Recharts · Leaflet
Infrastructure	Docker Compose · nginx · GitHub Actions
External services	Groq (LLM) · Brevo (email) · Cloudinary (files) · Typeform (applications)

The argument

One language on both sides: types are shared between server and client, which removes an entire class of interpretation errors.

Slide 6.2

Why these choices

Choice	Reason
NestJS	Modular structure and a guard mechanism that suits declarative access control, route by route
Prisma	Typed database access and migration tooling; queries are parameterised, so injection is not reachable
Ant Design	Covers back-office components without bespoke development
nginx in front	Serves the built application and proxies the API on the same origin: no CORS configuration at all
Configurable LLM client	The provider URL is a setting: changing provider is configuration, not a rewrite

Design

3 slides

Slide 7.1

Use case diagram

Image to insert

docs/diagrams/01-use-case-global.png

Portrait format (ratio 0.73). On a 16:9 slide, centre it under the title; do not stretch it.

What the diagram shows

- **9 actors** across the three tenancy levels
- **10 use cases** grouped by domain
- Two **generalisation** relationships: the tenant admin inherits from the employee, the client admin from the manager

Say this

Point at the receptionist: it is the role most systems do not have, and it exists only because a lease is concluded in person.

Slide 7.2

Class diagram

Image to insert

docs/diagrams/02b-class-diagram-presentation.png

A **landscape, reduced** version (ratio 1.96) built for projection: 10 classes, 3 attributes each. The full diagram (02-class-diagram.png, 17 classes) is unreadable on a screen; keep it for the written report.

What to point out

- The composition chain **Tenant → Building → Floor → Space**: this is the landlord's path
- The tenant_id attribute on **Booking** and **MaintenanceTicket**: this is the renter's path
- The note attached to Booking, which states the duality

Sequence diagram (pick one)

One sequence is enough. Two candidates, depending on what you want to prove.

File	What it shows	When to pick it
04-sequence-authorisation.png	The 5 authorisation stages of one request, with the refusal branches	For a technical panel: this is the core of the security model
06-sequence-booking.png	From public enquiry to signed lease, with the role driving each step	For a business-minded panel: the full operational journey

All nine diagrams are in docs/diagrams/, as PNG and as PlantUML source.

Say this

Do not project more than two diagrams. A panel disengages in front of a run of dense schematics; one well-explained diagram is worth more.

Progress

3 slides

Slide 8.1

The project in figures

25	30	55	9	252	43
data models	controllers	pages	roles	tests	audit probes

In Canva: six aligned tiles, the number very large with the label beneath. This is the slide that conveys the scale of the work.

Overall state

- Continuous integration green
- Type checking clean on both projects
- Full deployment from a single command

Slide 8.2

Work completed

Complete business chain

- Application → booking → lease → invoice → payment
- Maintenance, live notifications, audit trail
- Dashboards and analytics per role

Isolation

- All three levels in place and verified
- **9 defects found and fixed**
- Arithmetic consistency check (slide 5.6)

Artificial intelligence — beyond the specification

- Automatic ticket triage: category, priority, suggested technician
- A space-finding assistant for visitors, in plain language
- Reading of administrative documents (trade licence, commercial registration)

The principle adopted for AI

The model proposes, it does not decide. Any value outside the application's own enumeration is discarded and a keyword classifier takes over. The suggested technician comes from ticket history, not from the model.

Work not yet done

Outstanding item	State	Reason
Document-reading interface	Server only	Extraction works; the drag-and-drop into the form remains to be built
Scanned documents	Limited	The configured LLM provider exposes no vision model
External marketplaces	Waiting	Adapters written, credentials not supplied
Google Business profile	On hold	Requires a verified account held by the property owner
Production email	Degraded mode	Server IP to be allow-listed with the provider; SMTP fallback active
Mobile application	Out of scope	The web interface is responsive

Conclusion

None of these blocks a business journey: each has either a degraded mode or a manual equivalent already in place.

Say this

Do not play this down. A clear-eyed progress report carries more weight than a list where everything is declared finished.

Conclusion and perspectives

2 slides

Slide 9.1

The main finding (dark background slide)

Slide text

None of the nine defects came from a broken security function. Every one came from a call site that forgot to use it.

And four stayed invisible while the dataset held a single landlord: with one, “show everything” and “show my portfolio” return the same rows.

→ **The shape of your test data is part of the security surface.**

In Canva: background #084449, white text, no illustration.

Say this

The turning point of the defence. Deliver it slowly. This is an engineering result, not a feature list.

Slide 9.2

Assessment and perspectives

Assessment

- A platform that works from anonymous visitor to platform owner
- An isolation model that is **demonstrable**, not merely asserted
- AI that is useful and bounded: it proposes, it does not decide
- A deployment reproducible from one command

Perspectives

- Build the document upload interface, then move to a vision model for scans
- Extend the conversational assistant to the renter's own data, with a suite of injection tests to demonstrate its limits
- Replace the maintenance heuristic with a model trained on ticket history, measured against that heuristic as the baseline
- A mobile application for technicians in the field

Final slide: “Thank you for your attention”, then “Questions”.

Figures taken from the database and from a test run on 3 October 2026. The images referenced are in docs/diagrams/. Section 1 is left as a template: the host organisation and supervisor names appear nowhere in the code.